Computational Economics

Lab: Working with AI Agents

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

Lab Overview

- **Choose and direct an agent**
  - Model families, evaluation criteria, and agent applications
  - Tool execution, CLI, and MCP
  - A concrete task prompt
- **Manage the information it uses**
  - Context, project instructions, and reusable skills
  - Retrieval, compaction, saved notes, and caching
- **Check and reuse its work**
  - Inspect the result and rerun the program

LLM Families and Developers

A **large language model (LLM)** processes context and generates text, code, or requests to perform actions.

- **GPT family** — developed by **OpenAI**.
  Examples: GPT-6 Astra, GPT-6.1 Sol, GPT-6 Luna.
- **Claude family** — developed by **Anthropic**.
  Examples: Claude Fable 5, Claude Opus 5.5, Claude Sonnet 5.5.
- **Gemini family** — developed by **Google DeepMind**.
  Examples: Gemini 4 Argon, Gemini 3.8 Flash.
- **Grok family** — developed by **xAI**.
  Examples: Grok 4.20, Grok 4.7.
- **DeepSeek family** — developed by **DeepSeek**.
  Examples: DeepSeek-V4-Pro, DeepSeek-V4.1-Flash.

How LLMs Are Evaluated

LLMs are commonly judged along the following dimensions.

- **Task performance**: accuracy or success on one attempt (**pass@1**). For example, Terminal-Bench tests tasks performed with terminal tools.
- **Reliability**: factual errors, instruction-following failures, and consistency across repeated runs.
- **Token use and cost**: a **token** is a unit of text, often part of a word. Cost depends on token usage (including reported reasoning) and token prices; a useful measure is **cost per completed task**.
- **Speed**: time to the first answer token, output tokens per second, and total completion time. Longer reasoning can increase total time.
- **Input capabilities**: supported languages and **multimodal inputs**, such as text, images, audio, and video.
- **Long-context performance**: context-window size and accuracy when using long documents.

Scores depend on the test setup: extra attempts, additional tools, or more reasoning time can improve results while increasing cost.

The Harness: Tools and Interfaces

A **harness** is the software surrounding an LLM. It supplies tools, manages context and permissions, and returns tool results to the LLM.

- **Files and coding:** search, read, create, and edit files.
- **Execution:** run Python, terminal commands, and tests.
- **Web access:** search the web, retrieve pages, and operate a browser.
- **External services:** access a database, cloud drive, or other connected service.

Tools can be accessed through different interfaces:

- A **command-line interface (CLI)** operates software through text commands. The harness can run `git status` to inspect repository changes.
- **Model Context Protocol (MCP)** standardizes how applications connect to servers providing tools and information.

How the Harness Accesses Tools

The harness executes requests and supplies results as input to the next LLM call.

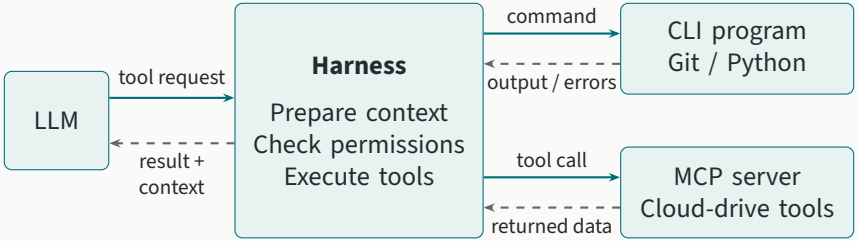

Figure 1: Solid arrows: requests. Dashed arrows: returned information.

- The **command runner** and **MCP client** are components used by the harness.
- CLI and MCP can work together in one task; an MCP server can itself run a CLI program. These are examples of tool access, not an exhaustive list.

The Agent: Coordinating a Task

An **LLM-based AI agent** pursues a task by selecting actions, using tools, and adapting its next actions to the results.

LLM + harness = AI agent

Agent applications: Codex (OpenAI), Claude Code (Anthropic), Cursor Agent (Cursor), Gemini CLI (Google), and GitHub Copilot (GitHub).

Example: a weekly temperature chart

You have: a shared-drive `data.csv` with `date` and `temperature_c`.

You want: a checked chart in date order, with Celsius units, and a reusable Python script.

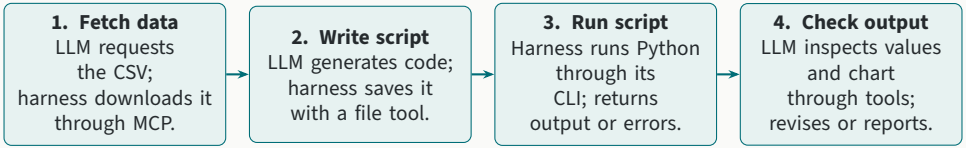

Figure 2: Read left to right: each step uses the previous step's result.

From a Vague Request to a Concrete Task

A concrete prompt makes the intended result and the criteria for success explicit.

The weekly temperature-chart task

Vague prompt: “Make a chart from my data.”

Concrete prompt

- **Input**: download the shared-drive `data.csv`, containing `date` and `temperature_c`, into the working folder.
- **Output**: create `plot_temperature.py` and save a line chart to `results/temperature.png`, with dates in order and Celsius units.
- **Constraints**: keep the input file unchanged and use the project Python environment.
- **Checks**: run the script, compare plotted dates and values with the CSV, inspect the chart, and report the command used.

These details give the agent an observable target. The checked script can be reused when next week’s CSV arrives.

The Prompt Is Part of the Context

Context is the input supplied to an LLM call, including more than the latest user message.

The chart exists; the user now says “Fix the axis label.”

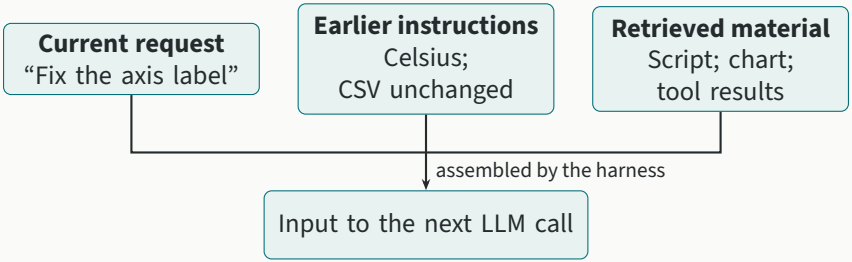

Figure 3: Selected instructions and material accompany the new request.

- Tool definitions and selected conversation history may also be included.
- The **context window** limits tokens per call; the full input may greatly exceed the new message.

Reuse Instructions and Procedures

Recurring guidance can be stored once and loaded into context when it applies.

- **Task prompt**: the current request.
  Example: “Create this week’s chart from the shared-drive CSV.”
- **Project instructions**: standing rules for work in the project.
  Example: preserve input data, label units, and save figures in `results/`. Codex supports `AGENTS.md`; conventions vary by application.
- **Skill**: a reusable procedure, with optional scripts and reference material.
  Example: a plotting skill describes how to read columns, choose axes, produce the chart, and check it. In the Agent Skills format, instructions live in `SKILL.md`.

Skill metadata helps the agent discover a relevant procedure; its instructions can then be loaded into context.

Skills guide the procedure; tools perform its operations. The plotting skill can guide the use of file tools and Python execution.

Managing Limited Context

Conversation history and tool outputs accumulate as work continues. **Context engineering** determines which information accompanies the next LLM call.

- **Retrieval**: load material relevant to the current step.
  For an axis-label fix, retrieve the plotting code and current chart; a full execution log may add little.
- **Compaction**: replace earlier conversation detail with a shorter summary.
  For the chart task, preserve the input location, Celsius units, script path, completed checks, and remaining work. A summary can omit useful details.
- **Saved notes**: retain decisions across sessions.
  A note influences a later LLM call when the harness retrieves it into context.

A concise handoff

"Shared-drive CSV downloaded. Script: `plot_temperature.py`. Keep dates sorted and use Celsius. Script runs; next check plotted values against the CSV."

Context Caching Reuses Computation

Context caching reuses computation for an unchanged beginning of the input—its **prefix**—when a compatible cache is available.

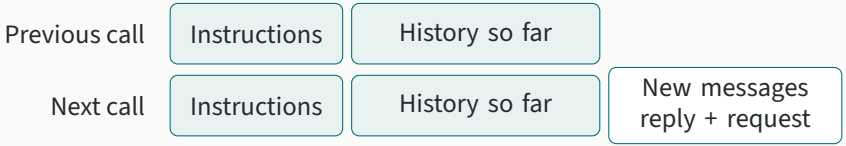

Figure 4: Unchanged prefix: reuse computation. New messages: process normally.

- Cached input can reduce cost and processing time; eligibility, prices, and retention depend on the provider.
- Changing the prefix can reduce reuse. Cached input still occupies context space; an expired cache requires recomputation.

Caching reuses computation; compaction changes the input. A summary may shorten context but invalidate a cached prefix.

Verify the Result

A script that runs successfully can still produce the wrong chart.

Example check: three observations in the CSV

Date	Temperature (Celsius)
2026-10-01	20
2026-10-02	22
2026-10-03	21

- **Data agreement**: three points in date order, with a peak of 22 on October 2; axis labels show dates and Celsius.
- **Input preservation**: the original CSV is unchanged.
- **Reproducibility**: rerunning `python plot_temperature.py` in the recorded environment regenerates the saved chart.
- **Reusable work**: the checked script and its run instructions are saved in the Git project.

References and Further Reading

1. **OpenAI, Agent and Codex documentation. [Documentation]**
   - Codex as a platform: build on the open agent harness
   - Prompt engineering
   - Prompt caching
2. **Anthropic, Claude model and Claude Code documentation. [Documentation]**
   - Claude Code: Overview
3. **Microsoft, Visual Studio Code: agent and terminal documentation. [Documentation]**
   - Understand agent harnesses
4. **Model Context Protocol contributors, Model Context Protocol documentation. [Documentation]**
   - Architecture overview

References and Further Reading

5. **Anthropic (2025), Effective context engineering for AI agents. [Technical article]**
   - Context engineering vs. prompt engineering
6. **Agent Skills contributors, Agent Skills documentation. [Documentation]**
   - Agent Skills Overview
7. **CarpeDiem (2026), AI Learning from Zero: practical agent tutorials. [Video series]**
   - Code is the most handy tool for AI
   - Agent cost-saving tips (Part 1): tokens and context
   - Agent cost-saving tips (Part 2): context caching
8. **Artificial Analysis, Benchmarking methodology. [Web resource]**